PIMPRI CHINCHWAD EDUCATION TRUST's

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING



Department of Computer Science and Engineering

(Artificial Intelligence and Machine Learning)

ARTIFICIAL INTELLIGENCE LAB MINI PROJECT REPORT

Retail Run

Smart Retail Store Automation Using Search Algorithms

Project Mentor and Guide

Prof. Pallavi Nikumbh Dhade

Submitted by

Student Name	PRN
Soham Mulik	124B1C134
Rudra Tere	124B1C127

Abstract

Retail Run turns a supermarket floor into an interactive planning problem. A simulated robot collects customer orders, replenishes shelves, guides shoppers, checks departments and returns to a charging dock. The application provides five navigation choices: breadth-first search, depth-first search, uniform-cost search, Greedy Best-First Search and A*. An autonomous controller selects work from battery level, shelf stock and a queue of incoming orders. Temporary trolley, spill and crate blockages modify the traversable map while a mission is running, causing the robot to compute another route or wait when no route exists. Separate workbench exercises demonstrate constrained robot allocation and worst-case service decisions. Verification of the seven-node FA1 graph reproduced path costs of 13, 16, 10, 10 and 10 units for the five methods. In two deterministic 120-second controller runs, 11 missions finished in each condition; enabling temporary blockages raised travel cost from 289 to 299 units and produced 15 replans. These small controlled checks establish the implemented behaviour, but do not measure performance in a physical retail store. The project connects classroom search concepts to visible operational decisions without training a machine-learning model.

Keywords: Intelligent agents, retail robotics, A* search, dynamic obstacles, autonomous tasks, constraint satisfaction, Alpha-Beta pruning.

Contents

Abstract i

1 Introduction 1

1.1 Project objectives . . . 1

1.2 Scope . . . 1

2 Simulation and Planning Problem 1

2.1 Rules of the retail simulation . . . 1

2.2 State, actions, and travel cost . . . 2

3 Search and Decision Methods 3

3.1 Navigation algorithms . . . 3

3.2 Constraints and service decisions . . . 3

3.3 Autonomous control loop . . . 4

4 Experimental Setup 4

5 Results 5

5.1 Search evaluation . . . 5

5.2 Dynamic-environment checks . . . 5

5.3 Reading autonomous decisions . . . 8

6 Discussion 9

6.1 Limitations and future work . . . 10

7 Conclusion 10

1 Introduction

A store robot rarely follows one route for an entire shift. An order may require a visit to a shelf and then the pickup desk; replenishment starts in the stockroom; a blocked aisle can invalidate either journey. The useful decision is therefore not just where to move, but which task to undertake and when to revise the plan.

Retail Run develops the Smart Retail Store Automation case study into a browser application [1]. The grid-and-robot presentation was inspired by the Rescue Run project [2], while the missions, inventory rules and search comparisons are specific to retail operations. Users can issue tasks directly or allow an autonomous controller to select them.

1.1 Project objectives

- Represent shelves, service locations, robot state and aisle costs in one executable store model.
- Implement five search strategies and compare their returned costs and expanded states.
- Demonstrate task execution, stock updates, charging and automatic responses to aisle blockages.
- Expose constrained task assignment and Minimax decisions through an explainable AI workbench.

1.2 Scope

The application models one moving robot on a fully visible discrete floor. Shelf stock, payload and battery are simulated values. It uses classical search and priority rules; it does not learn a policy from experience. The three-robot CSP and service-priority tree are separate demonstrations, not an integrated fleet controller.

In PEAS terms, performance concerns task completion, travel and valid stock handling; the environment is the store grid; actuators are movement and service actions; percepts are simulated inventory, orders, battery and blockages. Unlike the broader FA1 setting, the implemented map is fully observable and its scripted events are deterministic.

2 Simulation and Planning Problem

2.1 Rules of the retail simulation

Pip moves between adjacent cells in four directions. Shelf cells and marked obstacles are unavailable. The store has six departments, a stockroom, customer entrance, pickup desk and charging dock. Picking transfers goods from shelf stock to the robot, and delivery empties its carried load. Restocking transfers goods from the stockroom to the selected shelf.

Every item is assigned a weight of 1 kg, and Pip can carry at most 8 kg. A mission is rejected if its requested goods or payload are infeasible. The field guide explains the controls and the assumptions behind the simulation.

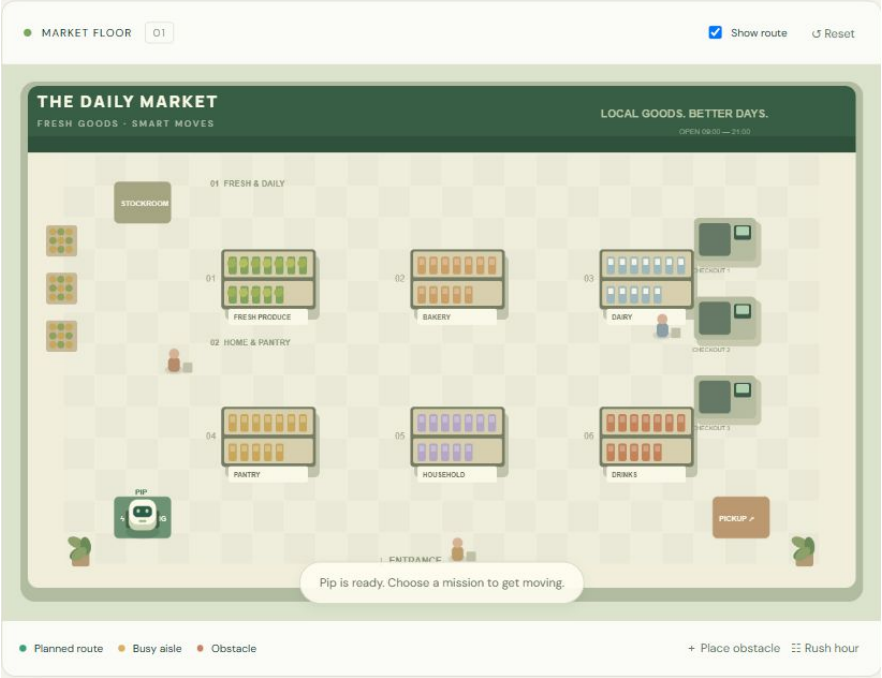


Figure 1: Retail Run store floor with departments, service stations and Pip at the dock.

2.2 State, actions, and travel cost

A navigation state identifies the robot cell. The operational state also contains inventory, carried goods, remaining battery, active task stages and the blocked-cell set. These values change as movement and service actions finish.

Table 1: Main elements of the implemented simulation.

Element	**Representation**
Floor	24 columns by 15 rows; rows 1 to 13 are navigable
Position	Cell identifier $y \times 24 + x$
Inventory	Shelf and stockroom counts for six departments
Tasks	Pick, restock, guide, inspect and charge
Move cost	1 normally; 4 in central rush-hour cells
Battery	0.18 percentage points per travel-cost unit

Listing 1: Cell indexing used in the application.

```
const W = 24, H = 15;
const key = (x, y) => y * W + x;
const xy = n => [n % W, Math.floor(n / W)];
```

3 Search and Decision Methods

The simulator separates route selection from task selection. Navigation searches the current weighted grid. The autonomous controller chooses a job, while the workbench illustrates allocation and service-planning problems. This separation makes the role of each AI method visible [3].

3.1 Navigation algorithms

Breadth-first search uses a queue and favours the fewest moves. Depth-first search pursues a branch before returning to alternatives. Uniform-cost search selects the lowest accumulated cost. Greedy Best-First Search considers only the estimated remaining distance. A* combines both quantities [4]:

$$f(n) = g(n) + h(n) \tag{1}$$

$$h(n) = |x(n) - x(goal)| + |y(n) - y(goal)| \tag{2}$$

Here, g(n) is the travel cost already incurred. Manhattan distance is a lower bound because moves are orthogonal and every available move costs at least one. Walls can lengthen a route but cannot make it shorter than this bound. A* and UCS stop when the goal is selected; the current map is treated as fixed during each search.

Listing 2: Priority score used by the search engine.

```
const score = x => method === 'UCS' ? x.g
  : method === 'Greedy' ? heuristic(x.n)
  : x.g + heuristic(x.n);
open.sort((a, b) => score(a) - score(b)
  || String(a.n).localeCompare(String(b.n)));
```

Successors on the grid are considered up, right, down and left. The engine stores parents, best costs and closed states. Its priority frontier is an array sorted before selection, rather than a binary heap; measured execution times would therefore reflect this implementation choice as well as the search strategy.

3.2 Constraints and service decisions

The allocation exercise contains three overlapping jobs of 7, 4 and 4 kg, and robots with capacities of 8, 5 and 7 kg. Backtracking rejects a robot that is already assigned, overloaded or below the battery threshold. With all three available, one feasible assignment is R1, R2 and R3. Removing R1 makes the three concurrent jobs infeasible.

The service exercise evaluates keep-plan, reroute and add-staff options against two demand outcomes. Minimax selects the largest worst-case score; Alpha-Beta skips outcomes that cannot improve the current choice. Staff availability removes an infeasible option before comparison. The displayed scores are illustrative FA1 inputs [1], not measured customer satisfaction.

3.3 Autonomous control loop

Autopilot makes a decision only when Pip has no active mission. It first checks whether battery is below 35%. Otherwise, a shelf with four or fewer units is replenished when reserve stock exists. Pending customer orders are considered after these checks. A shortage for the next order triggers replenishment; an order without any replenishment stock is reported unavailable.

Listing 3: Simplified decision order implemented by Autopilot.

```
if (missionIsActive) return;
if (battery < 35) return chooseCharging();
if (lowShelfHasReserve) return chooseRestocking();
if (nextOrderHasStock) return choosePicking();
if (nextOrderNeedsReserve) return chooseRestocking();
return waitOrReportUnavailableOrder();
```

This listing summarizes the controller rather than replacing its validation logic. The dispatcher still checks the route, quantity and energy needed for the planned journey and a return to the dock. Failed dispatches are retried after a short controller interval. Turning Autopilot off allows the current mission to finish.

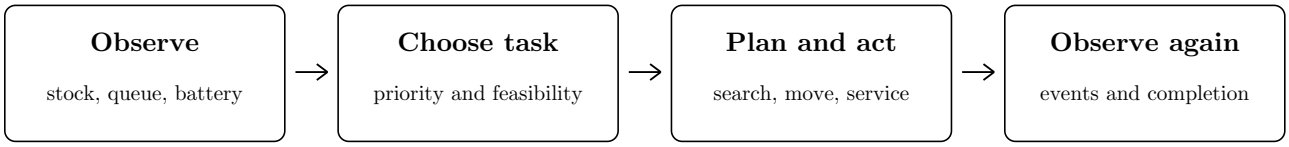


Figure 2: Repeated observation, task selection, planning and action in Autopilot.

4 Experimental Setup

The checks used the JavaScript source at commit 91ff0492d196923db9319ccb016650f9a54fbe4a, executed with Node.js v24.22.1 on 27 October 2026 [5]. A headless harness supplied the document controls used by the application and advanced its existing tick function. It did not replace the route planner or mission-update functions.

Three sets of checks were run: the seven-node FA1 graph; six dock-to-department queries on the store grid, under normal and rush-hour costs; and two 120-second autonomous runs with identical starting orders. The autonomous comparison changed only whether temporary environmental events were enabled. These are repeatable examples, not samples from a population of stores.

Listing 4: Controlled stepping for one autonomous run.

```
reset();
auto.enabled = true;
auto.environment = eventsEnabled;
auto.orders = [{id: 0, qty: 2}, {id: 2, qty: 2}];
for (let i = 0; i < 1200; i++) tick(100);
```

The run used 1× speed and 100 ms increments. New orders arrived every 12 event-clock seconds. Live-environment events began after six seconds, repeated every 14 seconds and expired 18 seconds after insertion. These intervals belong to the event clock; the on-screen store clock advances separately with completed travel and service steps.

- Travel cost: sum of movement weights, excluding service dwell and charging time.
- Expanded states: selected non-goal states whose successors were explored.
- Completed missions and replans: counters recorded from the live simulation state.

5 Results

5.1 Search evaluation

The FA1 verification reproduced all five expected route costs. BFS and DFS did not return the minimum-cost route on this weighted graph. Greedy matched UCS and A* for this example, but its decision rule does not provide a general optimality guarantee.

Table 2: Verified FA1 routes from stockroom S to pickup G.

Algorithm	**Returned route**	**Cost**	**Expanded**
BFS	S-A-C-G	13	6
DFS	S-A-C-D-E-G	16	5
UCS	S-B-D-G	10	6
Greedy	S-B-D-G	10	3
A*	S-B-D-G	10	3

Across the six store-grid destinations, BFS, UCS, Greedy and A* each totalled 81 normal-cost units and 141 rush-hour units. DFS totalled 927 and 1,557 units because its traversal order produced long routes. Equal totals on this map do not establish that BFS or Greedy will match cost-sensitive search on other layouts.

5.2 Dynamic-environment checks

A controlled event placed an obstacle on the planned produce-collection route before movement began. The route changed and the replan counter increased to one. Both routes cost 10 units: the nearby aisle provided an equally short detour. A visible environmental response therefore need not increase the final path cost.

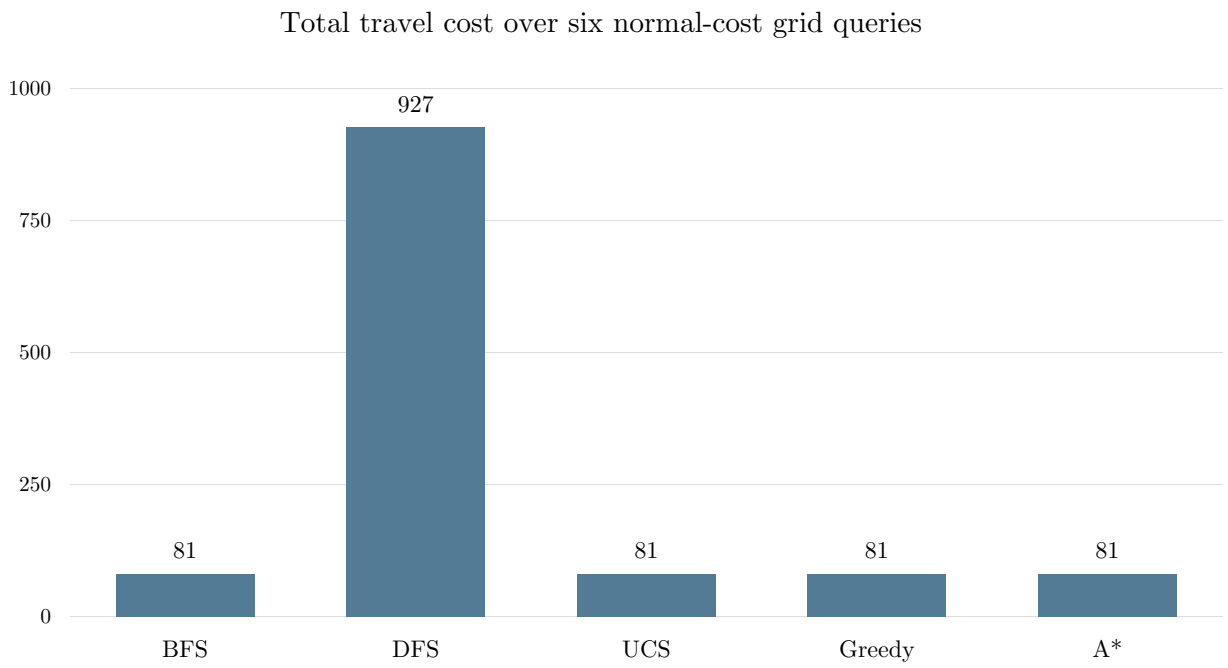


Figure 3: Sum of returned route costs across six department destinations, starting at the dock.

DFS is included as a traversal baseline. Its unusually long routes are a consequence of this map and the fixed successor order. It remains a valid reachability method here, but is unsuitable when the practical aim is short travel. A different traversal order could change these results substantially.

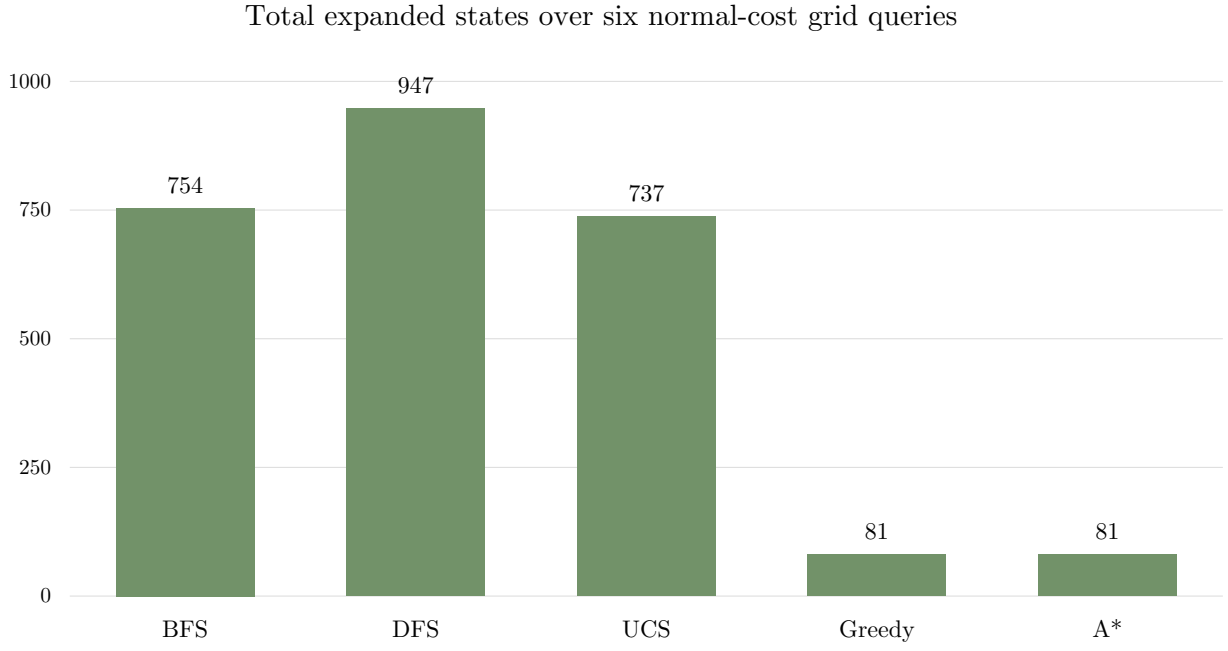


Figure 4: Non-goal expansions for the same six normal-cost queries.

A* expanded 81 states across the six queries, compared with 737 for UCS. The heuristic was informative on these particular routes. This count measures search work, not animation time, and excludes the goal state. It should not be interpreted as a universal speedup factor.

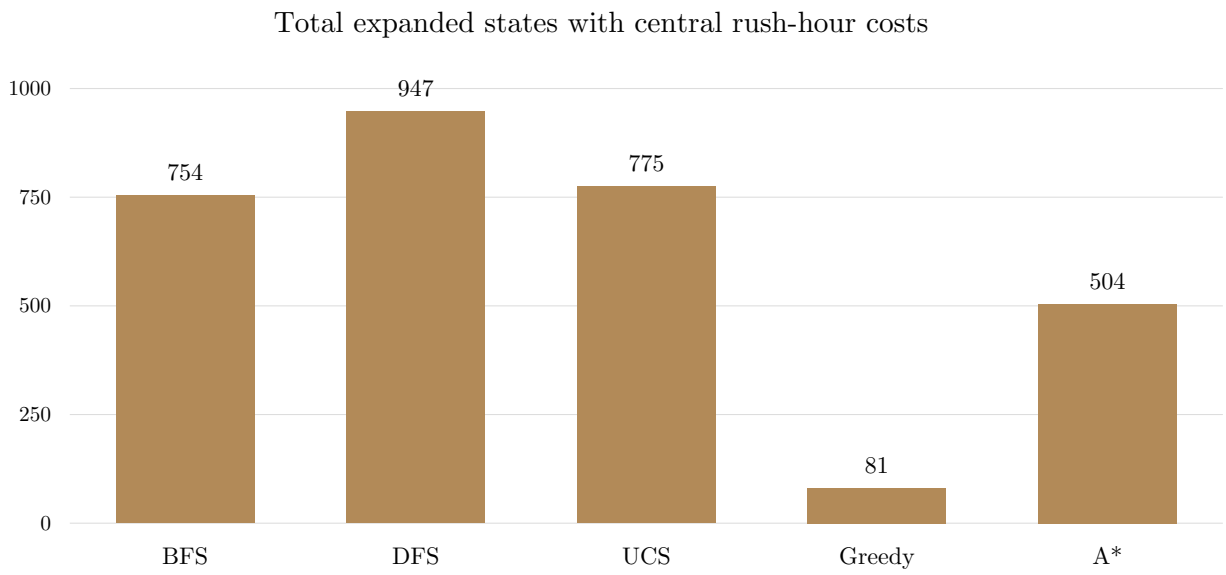


Figure 5: Expanded states after changing the central aisle entry cost from 1 to 4.

A* expanded more states under rush-hour costs because unit Manhattan distance became a weaker estimate of the remaining weighted cost. UCS still returned the same minimum costs. Greedy retained a low expansion count in this test; this efficiency is not evidence that its routes would remain optimal after arbitrary cost changes.

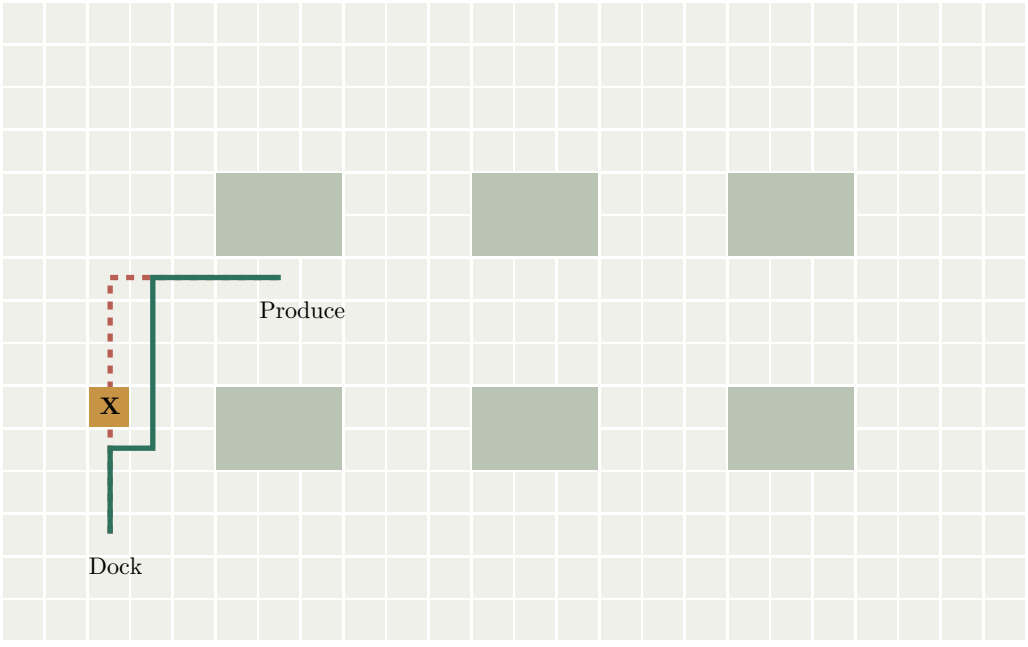


Figure 6: Produce route before and after a temporary blockage. Dashed red is the old route; green is the replacement. The amber cell is unavailable.

The blocked cell was (2, 9), using zero-based grid coordinates. Pip avoided it through the adjacent column. The diagram was reconstructed from the paths returned by the application, rather than drawn as an assumed route.

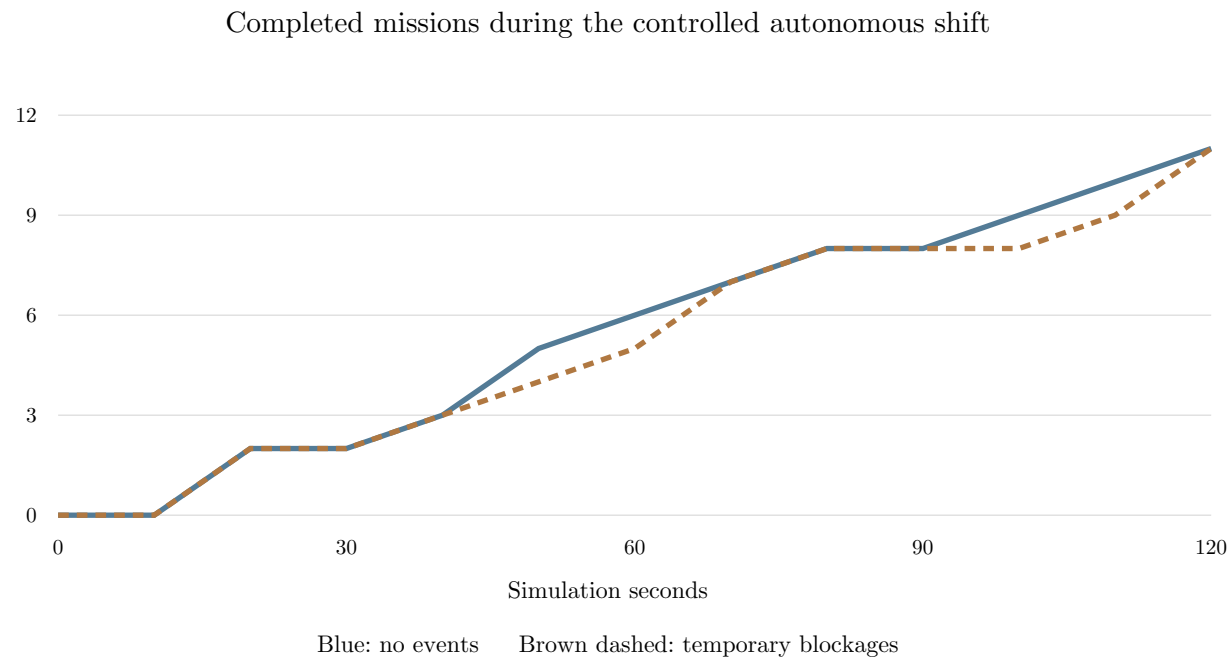


Figure 7: Completed missions sampled every 10 seconds in two deterministic 120-second runs.

Table 3: Autonomous run outcomes with A* navigation.

Measure	**No events**	**Temporary blockages**
Completed missions	11	11
Travel cost	289	299
Replans	0	15
Battery remaining	47.98%	46.18%
Queued orders at end	0	0

5.3 Reading autonomous decisions

The two runs processed the same initial orders and periodic arrivals. At the end, neither had an active mission or a queued order. Their shelf counts were also equal: 6, 8, 6, 10, 10 and 10 units in department order. Travel increased by 10 units, approximately 3.46%, when blockages were enabled, while the completion count stayed unchanged.

These two runs did not reach the low-stock or charging thresholds, so they do not by themselves verify those branches. Separate tests started Dairy at three units and battery at 25%. The controller selected stockroom collection in the first case and a charging mission in the second. This distinction prevents an unexercised feature from being reported as an observed outcome.

The reason label states why the next mission was selected. The environment journal names each obstruction and records reopening. A coral route trace briefly retains the previous plan, while the green trace identifies the replacement. These displays help relate the search decision to the floor change.

Table 4: Representative implementation checks and observed outcomes.

Check	**Observed outcome**
Pick and deliver 2 apples	Shelf 12 to 10; load 0 after delivery
Restock the same shelf	Shelf 10 to 12; reserve 30 to 28
Customer guidance and inspection	Both mission types completed
Pause with events enabled	Event-clock value remained unchanged
Temporary obstruction	Route changed; expiry removed the blockage
Capacity and stock validation	Oversized basket and unavailable stock rejected
Autopilot priorities	Low stock chose restocking; 25% battery chose charging
Reset	Autopilot off; temporary blockages removed

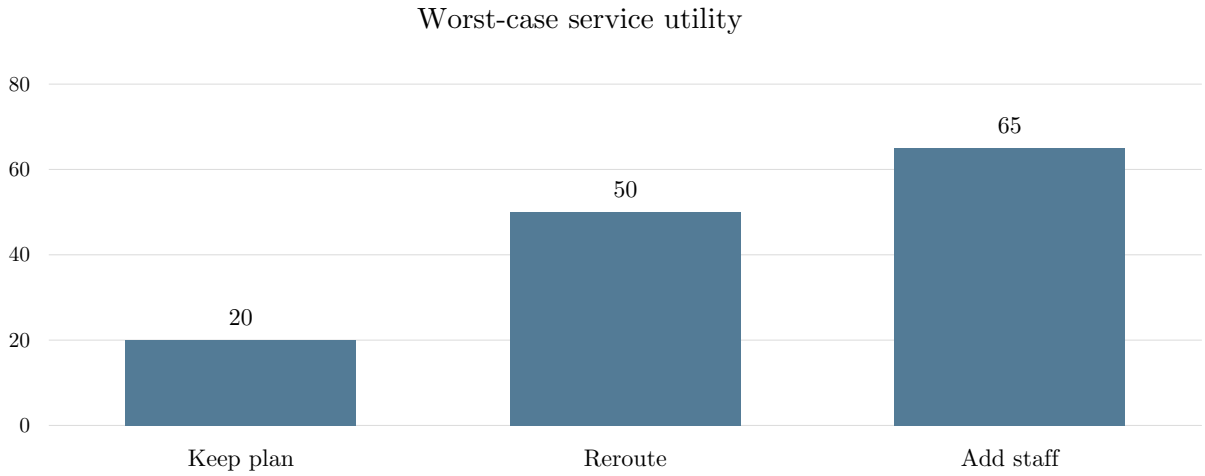


Figure 8: Service decision values in the workbench; larger worst-case utility is preferred.

For the supplied utility values, both Minimax and Alpha-Beta select the staff option with value 65. Evaluating that option first lets Alpha-Beta inspect four of six leaves and prune two. In the CSP exercise, R1/R2/R3 is feasible; with R1 unavailable, backtracking exhausts the remaining assignments and reports infeasibility.

6 Discussion

Retail Run makes several otherwise abstract distinctions observable. A route may change without becoming longer; a heuristic may reduce expansions on one cost map but offer less benefit on another; and a feasible task assignment can disappear when one resource is removed. The application exposes these effects through paths, stock changes, counters and decision traces.

The autonomous controller removes repeated manual dispatch while remaining simple to inspect. Its priorities are deliberately fixed. It is not a learned policy or a global optimizer, and its decisions should be evaluated against the rules it actually implements.

6.1 Limitations and future work

- Experiments use one synthetic store, six destinations and one deterministic event sequence. They do not establish reliability across arbitrary layouts.
- The robot sees the entire blocked-cell map immediately. Decorative shoppers do not create independently sensed motion obstacles.
- CSP allocation and service Minimax are workbench exercises. The floor controller does not execute simultaneous multi-robot schedules or apply the Minimax choice.
- The delivery-slot narrative is explanatory text; shared-aisle reservations, MRV and forward checking are not implemented.
- Travel cost, event time and the displayed store clock are different quantities. Battery use and item weight are simplified assumptions.

Further development should test more maps and event schedules, couple scheduling to actual robot motion, and add limited sensing. A longer-duration study could measure queue delay, stockouts and charging availability. A heap-based frontier would also permit a more useful runtime comparison than the current array implementation.

7 Conclusion

Retail Run implements a working connection between retail missions and classical AI search. Pip can carry out user commands or select orders, replenishment and charging tasks automatically. Temporary aisle blockages trigger replanning, and the interface shows the resulting route changes. Controlled tests reproduced the FA1 benchmark and completed 11 missions in both 120-second conditions, with 15 replans when events were active. The results support the behaviour of this classroom simulator; broader operational claims require additional experiments.

References

[1] S. Mulik, "Smart Retail Store Automation," Artificial Intelligence Formative Assessment I, Case Study 13, PCCOE, academic year 2026–27. Supplied project report.

[2] M. Vadapure, S. Suryawanshi and S. Kolhe, "Rescue Run: An Interactive Rescue-Robot Fire Escape Game using A* Search Planners," AI Lab project report, PCCOE, academic year 2026–27. Supplied reference report.

[3] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2020. Author resource: https://aima.cs.berkeley.edu/.

[4] P. E. Hart, N. J. Nilsson and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968, doi: 10.1109/TSSC.1968.300136.

[5] S. Mulik and R. Tere, "Retail Run," project application, source commit 91ff0492d196923db9319ccb016650f9a54fbe4a, inspected 27 October 2026. Supporting files: engine.js, app.js, autonomy.js, tests.cjs and simulation-tests.cjs.